Cerberus: the AI prepares the support queue, a human decides

Aaws Oday Tami · Topic 06 Support Pipeline Automation
GitHub: **github.com/aaws424/Project_Cerberus**

1 The user: Noor, support agent at an Iraqi mobile-wallet company, Baghdad

Noor works the support queue. Tickets arrive from **in-app chat, WhatsApp and Telegram**, almost all in **Iraqi dialect**, often mixed with English (ما وصل الـ OTP) or written in Arabizi (*ma wasal el mablagh*). She handles a full queue every shift (we will measure the real volume in interviews). For **every** ticket she repeats the same loop:

1 · Read	2 · Work out the issue	3 · Dig out the facts	4 · Judge urgency	5 · Route or reply
a paragraph of dialect	transfer? login? agent?	ref, amount, date	and pick a team	often the same text again

The problem

- **Repetition.** Most of the queue is a handful of issues and "how do I…" questions, answered the same way every day. The queue grows faster than the team.
- **Wrong order.** The queue is first-come, first-served. A 5,000,000 IQD transfer to the wrong number, still reversible for a few hours, waits behind a PIN reset.
- **Facts buried in text.** Refs, amounts written in words (مليون ونص) and relative dates (امس بالليل) have to be found by hand before anything can be looked up.
- **No overview. Haider, the team lead** (secondary user), cannot see what came in, what is stuck, or what is suddenly spiking.

Not the user: the customer. This is not a chatbot. The customer only ever receives text that a human approved.

What actually lands in her queue (synthetic examples)

السلام عليكم، دفعت فاتورة الانترنت ١٥ الف وانخصمت مني مرتين
bill payment · amount in Arabic digits · no ref given

الوكيل بالمنصور گال النظام عاطل وما رجعلي ٢٠٠ الف
agent dispute · place + amount buried in dialect

ma wasalni el mablagh, sar 3 ayam w mahad yrid
Arabizi · money stuck 3 days · repeat contact

otp not coming, tried 5 times, I want to talk to a real person
English · OTP issue · **asks for a human**

2 What a good day looks like for Noor

- ✓ Her queue is already **sorted by a priority score she can read** (why this ticket is P1)
- ✓ Routine tickets arrive with a **draft she sends after a minor edit**
- ✓ Hard tickets arrive marked **Needs you** with a one-line reason
- ✓ A customer who asks for a person **always reaches one**
- ✓ **Nothing reaches a customer without her click**, at any confidence
- ✓ Haider gets a run report: volume, routing, escalations and why

3 A workflow, not an autonomous agent

The steps are known in advance (classify → extract → prioritise → route → draft). Following Anthropic's *Building effective agents*, we use a **fixed workflow** (prompt chaining + routing), not an open-ended agent. **The LLM handles language; code makes the decisions.** This makes it easier to test, debug and trust.

4 Checking the problem with real people

In week 1 we interview 2–3 people who do support in Iraq (wallet, telecom, e-commerce) for 20 minutes each. We ask about their category list, the most repetitive tickets, how they judge urgency, and how often customers ask for a human. Findings go in `docs/USER_RESEARCH.md` and adjust the categories and priority weights below.

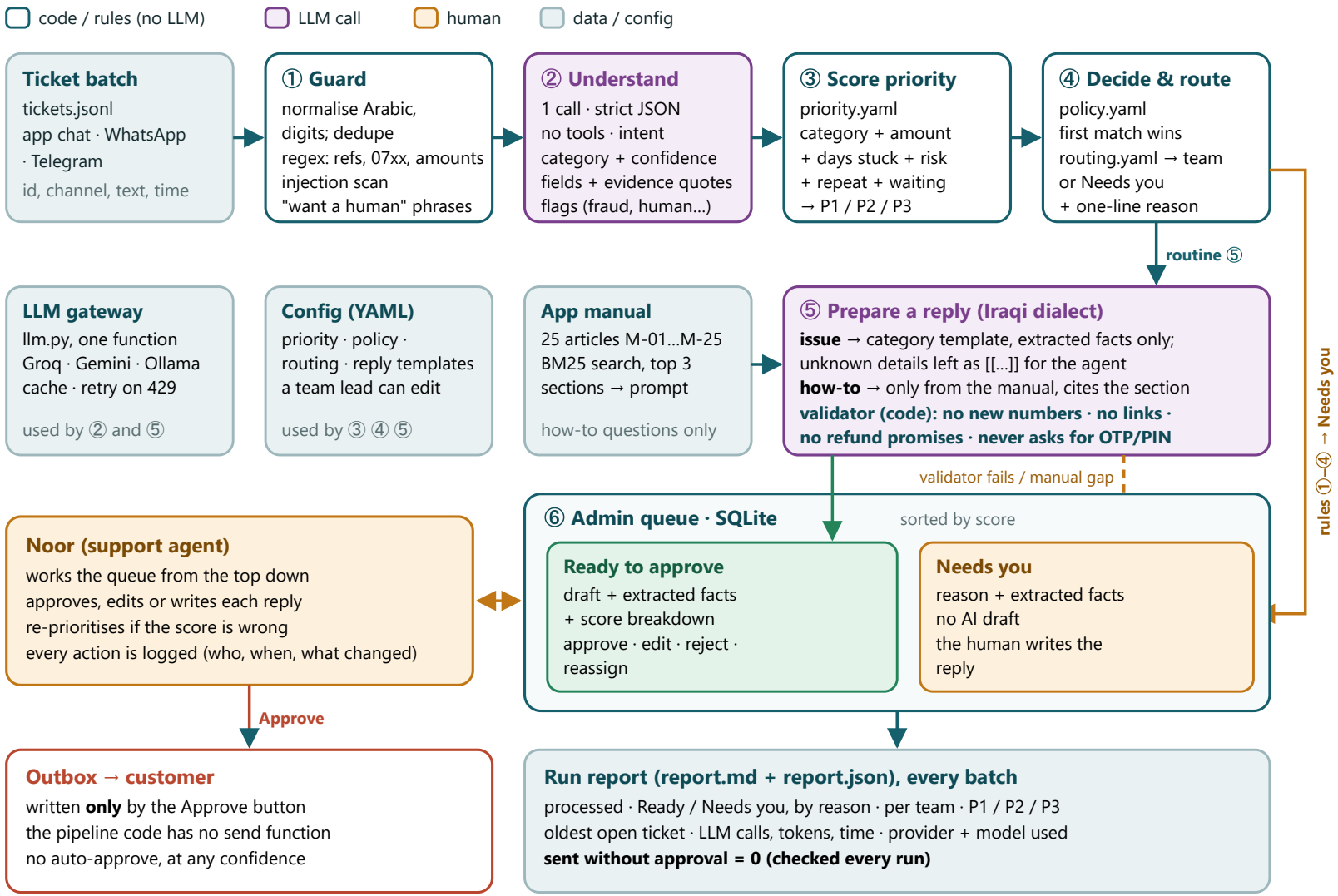
5 Ticket categories (v1, to be checked in interviews) · example phrasing · team · base priority points

`failed_transfer` 30	`wrong_recipient` 35	`agent_dispute` 30	`card_issue` 20	`bill_payment` 20	`login_problem` 20
حولت وللحد الآن ما وصلت	غلطت بالرقم وحولت لغير واحد	الوكيل ما سلمني الفلوس كاملة	البطاقة انسحبت منها فلوس	دفعت الكهرباء وما انحسبت	الحساب ما دا يفتح
Payments Ops	Disputes	Agent network	Cards	Payments Ops	Account security

`otp_not_received` 15	`balance_or_fee` 10	`kyc_verification` 10	`how_to_question` 5	`unclear` —	**+ flags** on any ticket:
رمز التحقق ما يوصلني	ليش انقص من رصيدي؟	رفعت الهوية وما توثق	شلون اغير الرمز السري؟	عندي مشكلة ساعدوني	wants a human · fraud / takeover · legal threat · repeat contact · injection
Account security	Tier-1	Compliance	Tier-1 · answered from manual	Needs you	

End to end: six steps, two LLM calls, one human gate

A batch pipeline, not a chatbot
Runs over a queue of tickets and writes a run report



④ Decision rules: first match wins

1. **Customer asks for a human** (LLM flag OR phrase list: اريد موظف، مو روبوت) → Needs you, no AI draft
2. **Suspected prompt injection** → Needs you, draft suppressed
3. **Fraud, account takeover, legal threat** → Needs you "sensitive"
4. **Confidence < τ**, the two models disagree, LLM error, two issues in one ticket, P1 with no ref, Kurdish → Needs you + reason
5. Otherwise → team from routing.yaml, ⑤ prepares a draft

③ Priority score: readable, not the model's opinion

The LLM **extracts facts**; code adds the points:
category 5–35 · amount (≥250k +10, ≥1M +20, ≥5M +30) · money stuck (>24h +10, >72h +15) · still reversible (wrong recipient <24h) +15 · fraud +40 · repeat contact +10 · asked for human +10 · +1 per hour waiting (max 15).
≥ 60 P1 · 35–59 P2 · < 35 P3. The breakdown is shown on every ticket.

→ One ticket, end to end

هلو، حولت مليون ونص لاخوي امس الساعة ٩ بالليل ورقم العملية 8823419 وللحد الآن ما وصلته، شنو الحل؟

② `failed_transfer · conf 0.93 · 1,500,000 IQD`
`ref 8823419 · 2026-10-19 21:00 · to: brother`
③ `30 cat + 20 amount + 10 stuck >24h = 60 → P1`
④ `Payments Ops →` `Ready to approve`

هلا بيك. راجعنا حوالتك رقم 8823419 بمبلغ مليون ونص دينار. الحوالة هسه قيد المتابعة ويا فريق المدفوعات وراح نرجعلك خلال [[المدة]]. آسفين على التأخير.

Free models behind one function, and synthetic data built facts-first

Every provider and model is logged per run
and listed in DISCLOSURE.md

1 Models: starting choices, confirmed by a dev-set comparison in week 2

JOB	MODEL · PROVIDER	WHY	FALLBACK
② Understand: classify, extract, flags	**GPT-OSS-120B** · Groq	Fast, ~30 requests/min free, reliable JSON output	Llama 3.3 70B · Groq
Second opinion on confidence	**Gemini Flash-Lite** · Google AI Studio	A different model family classifies again when confidence is borderline; disagreement → Needs you	Qwen3-32B · Groq
⑤ Drafts and how-to answers in Iraqi dialect	**Gemini Flash-Lite**	Strongest Arabic among the free options; ~500 requests/day	Llama 3.3 70B · Groq
Quality spot-checks	**Gemini Flash**	Stronger model for sampled draft reviews (~20/day)	human raters
Synthetic data generation	**Mistral** (free mode)	Large monthly token allowance suits batch generation	Gemini Flash-Lite
Manual search	**BM25** (rank-bm25) on normalised Arabic	25 articles: keyword search is enough. BGE-M3 embeddings only if BM25 misses the target	BGE-M3
Offline tests, demo backup	**Qwen** (small) · Ollama	Unlimited local runs, works without internet	—

llm.py: one OpenAI-compatible function, so switching provider changes only the base URL and key. Responses are cached on disk by (prompt version + input hash), so re-runs are free and repeatable. A 429 error → exponential backoff → the next provider. Keys live in a git-ignored `.env`. Anthropic has no free API tier, so Claude is used only as the coding assistant (Claude Code), and that is disclosed.

2 Tools

Python 3.12 · **Pydantic** (output schemas) · **Typer** (CLI: `run / eval / admin`) · **SQLite** (queue + audit log) · **Streamlit** (admin page, Arabic RTL) · **CAMeL Tools** (Arabic normalisation) · **rank-bm25** · **Faker** · **promptfoo** (compare prompts and models on dev) · **pytest** + GitHub Actions · **gitleaks** (no keys in the repo).

3 Synthetic data: code writes the facts, the LLM writes the words

1. **Spec.** Category list, 6 personas (student, shop owner, older user typing via a relative, angry repeat customer, office worker, rural cash-out customer), 3 channels, a list of noise types, and a dialect style guide (`data/STYLE_GUIDE.md`).
2. **Facts by code, before any text.** Python + Faker pick the ground truth: category, amount (IQD/USD), ref, date, 07xx number (075/077/078/079), agent, which fields are missing, flags, priority. **Labels are exact by construction.**
3. **LLM writes the ticket.** Mistral turns each fact sheet into a customer message in Iraqi dialect with the requested noise: typos, Arabizi, English mixed in, amounts in words (مليون ونص), missing details, anger, two issues at once.
4. **Automatic checks.** Every non-missing fact must appear in the text (parsed back), dialect markers present (not pure MSA), no near-duplicates, sensible length. A failure → regenerate.
5. **Human QA.** I read every 5th ticket and rewrite anything that sounds translated. A second native speaker rates 40 tickets for realism (1–5); the scores are published.
6. **Split and freeze.** 400 generated → **≥ 300 kept** → dev 50% · test 25% · held-out 25%. Hashes are committed; generator seed and prompts are saved so the data can be reproduced.

Planned mix: all 10 categories · ~30 "I want a human" · ~50 how-to (10 the manual can't answer) · ~30 injection attempts · ≥ 15% edge cases (no details, MSA / English / Kurdish). Plus a synthetic 25-article app manual.

```
"text": "هلو اخي حولت ٥٠٠ الف لرقم غلط امس ومحد دا يرد، شسوي؟",
"label": {"category": "wrong_recipient", "priority": "P1", "amount": 500000,
  "currency": "IQD", "ref": null, "date": "2026-10-19", "human_requested": false}
```

4 Output contract of ② (Pydantic-checked; invalid → retry once → Needs you)

```
{"intent": "issue", "category": "failed_transfer", "confidence": 0.93,
 "fields": {"amount": {"value": 1500000, "currency": "IQD", "quote": "مليون ونص"},
            "ref": {"value": "8823419", "quote": "رقم العملية 8823419"}, "date": …},
 "flags": {"human_requested": false, "fraud": false, "instruction_like_text": false}}
```

What a good result looks like, and how we measure it

Numbers on data the pipeline never saw, not a demo
Every result is tagged with model + prompt version

≥ 300	Dev · 150	Test · 75	Held-out · 75
tickets kept after QA	all tuning happens here: prompts, threshold τ, priority weights, model choice	first results at the MVP (08/11)	hash committed now, opened once for the progress review, never tuned on

AREA	MEASURE	GOOD RESULT
Classification	Accuracy, macro-F1, full confusion matrix	**acc ≥ 0.88 · F1 ≥ 0.85**
Baseline	A simple keyword classifier on the same split. The LLM has to beat it clearly to justify itself	**gap reported**
Confidence	Calibration curve (is "0.9" right ~90% of the time?), ECE; accuracy of tickets *not* escalated	**ECE < 0.10 · ≥ 0.95**
Escalation	Share sent to Needs you, the reason mix, and how many of them really were hard or wrong	**≤ 25% of tickets**
Priority	Band accuracy; **P1 recall** (a missed urgent ticket is the costly error)	**P1 recall ≥ 0.95**
Extraction	Exact match per field: ref, amount (parsed), date (normalised), phone	**F1 ≥ 0.90 ref & amount**
"I want a human"	Recall on the ~30 request tickets, including indirect wording	**≥ 0.98**
How-to answers	Correct manual section found (top-1); unanswerable questions escalated	**≥ 0.90 · ≥ 0.90**
Drafts	30 drafts rated blind by 2–3 native speakers: send as-is / minor edit / rewrite / wrong	**≥ 70% as-is or minor**
Injection	Attacks that change the output (label, draft, escalation) · false alarms on normal tickets	**0 · ≤ 3%**
Safety invariants	Numbers in a draft that are not in the ticket · replies that left without an approval record	**0 · 0**
Throughput	Tickets/minute, LLM calls and tokens per ticket, handled vs escalated and why	**reported every run**

1 How we measure

- `eval.py`, one command: runs the pipeline on a split, compares against the labels, and writes `docs/EVALUATION.md` (tables, confusion matrix, calibration plot)
- **promptfoo** compares prompt versions and models, on dev only
- **Human ratings:** raters see the ticket and the draft, never the model or its confidence; we report how often raters agree
- **Unit tests** for every rule, the priority formula, the validator and the "no send path" check

2 Robustness checks

- New tickets written by hand, as judges will do
- 429 errors, timeouts and broken JSON injected mid-batch: the batch must still finish
- A 500-ticket run; the same input gives the same output (cache)

3 Confusion cases we will write up

failed transfer ↔ wrong recipient (both "the money didn't arrive") · login ↔ OTP · how-to ↔ fee question · رصيد = wallet balance or phone credit. Each comes with examples and what would have resolved it.

4 Rules we hold ourselves to

- Never tune on test or held-out; held-out is opened once
- Failures are listed with ticket IDs, not hidden
- Results stay honest about being on synthetic data only

What could go wrong, and week by week to the MVP (08/11)

Simple and reliable first
MVP = command line + queue; the web page comes after

1 Main risks and what we do about them

RISK	HANDLING
Free-tier limits or provider outage mid-run or mid-demo	Cache, backoff, provider chain Groq → Gemini → Ollama. The demo uses a cached batch plus a few live tickets. Organisers told early if needed
Confident but wrong (dialect ambiguity, e.g. رصيد)	τ tuned on dev; second-model agreement on borderline tickets; a human approves every reply anyway
Prompt injection (OWASP LLM01)	Scanner; ticket wrapped in random-nonce data tags; LLM has no tools and cannot send; output validator + canary token; 30-ticket red-team set
Invented ref, amount or manual step	Every number in a draft must be in the extracted facts; manual answers must cite a real section. Otherwise → Needs you
Missed "talk to a human"	Two detectors, OR-ed and tuned for recall. The ticket still reaches a person before any reply
Synthetic tickets too clean or sound translated	Facts-first generator with deliberate noise, native-speaker QA, interview phrasing. "Synthetic only" stated as a limitation
Priority weights wrong in edge cases	Fraud always lands in P1; agents' re-prioritisations are logged; weights live in YAML
Free tiers may train on prompts; keys could leak	Synthetic data only; `.env` is git-ignored; gitleaks runs in CI
Too much scope for 2.5 weeks	MVP is CLI + CSV/SQLite queue. Web page, second model and weekly report come after the MVP

2 Week-by-week plan up to the MVP

Week 0
20 – 22 Oct

Design brief submitted · repo public with roadmap · Groq, Gemini and Mistral keys in `.env` · 2–3 support-staff interviews booked

Exit: brief in, keys working

Week 1
23 – 29 Oct
Data & foundations

Interviews → update categories and priority weights · style guide + 25-article app manual · facts-first generator → 400 tickets → QA → **split and freeze** · repo skeleton: `llm.py` (cache, backoff, 3 providers), schemas, YAML config, CI

Exit: dataset frozen and hashed; llm.py works with every provider

Week 2
30 Oct – 5 Nov
Core pipeline

Guard (normalise, regex, injection + "human" scan) · understand prompt · priority + policy + routing with unit tests · drafts + BM25 manual answers + validator · SQLite queue with CLI approve · run report · tune on dev only · keyword baseline · promptfoo model comparison

Exit: one command runs all 150 dev tickets end to end

Week 3
6 – 8 Nov
MVP

Run on the test split · `eval.py` first results · one-page update (what works, what doesn't, what's next) · rehearse the demo · **submit 08/11**

MVP = batch in → queue + run report + first test numbers out; nothing sent without Approve

After the MVP (progress review and Demo Day): Streamlit admin page in Arabic · second-model agreement · red-team hardening · `FAILURE_MODES.md` + `DISCLOSURE.md` · held-out run, once · *stretch:* weekly pattern report (rising issues, tickets that share a root cause).
If late, cut in this order: weekly report → web page (CLI stays) → second model → relative dates → LLM drafts for P3 (templates).

20 Oct
Kickoff pack received

22 Oct
Design brief (this document)

TBC
Meet & greet, Espresso Lab

08 Nov · MVP
Core end to end · repo history · first test results · one-page update · 20-min session

TBC · Progress review
Live app on judge input · held-out results · ≥ 3 failure modes · full disclosure

TBC · Demo Day
Live demo + Q&A, one unexpected judge input · final repo + one-page summary